8295/8255/MTK/8538平台CarPower接口

1、概要
2、电源状态机
2.1 CarPowerManager中的电源状态机定义
2.2 SettingsProvider中的电源状态机定义
3、对上层应用的接口
3.1 Google原生AndroidCar接口
3.1.1 连接AndroidCar服务
3.1.1.1 Car.java
3.1.1.2 Car.CarServiceLifecycleListener
3.1.2 CarPowerManager
3.1.2.1 CarPowerManager.java
3.1.2.2 CarPowerManager.CarPowerStateListener
3.1.2.3 CarPowerManager.CarPowerStateListenerWithCompletion
3.1.2.4 CarPowerManager.CarPowerPolicyListener
3.1.3 CarPowerPolicy
3.1.4 CarPowerPolicyFilter
3.1.4.1 CarPowerPolicyFilter.Builder
3.1.5 PowerComponent
3.2 镁佳ContentObserver方式监听电源状态机变化
3.3 Android侧强行切换整机电源状态机
4、对**Android** 应用层**app**的要求

变更记录

版本号	日期	变更描述	编制人
1.0	2024-05-16	初稿、正式发布	刘鑫
1.1	2024-07-01	更新PowerComponent新增定义	刘鑫
1.2	2024-11-25	补充强行切换电源管理状态机的方法	刘鑫
1.3	2025-11-26	更新电源管理状态机新增定义	刘鑫

1、概要

8295/8255/mtk/8538/8797平台Android 13/14/15/16上Google对CarPower电源管理做了部分改动，涉及到电源状态机和接口变更。因此在8295/8255/mtk/8538/8797平台Android 13/14/15/16上Android侧app对接电源管理使用sdk时，请参考该文档。

2、电源状态机

同8155平台Android 11 一样，我们在Google原生基础上增加了ContentObserver方式监听电源状态变化的方式，并将SettingsProvider中的状态机定义和Google原生CarPowerManager中的状态机定义保持一致。

2.1 CarPowerManager中的电源状态机定义

Google原生CarPowerManager中的电源状态机定义声明在CarPowerManager.java中，在**Google原生基础上镁佳平台增加了一些镁佳平台的状态机(绿色部分，下同)**，具体如下：

阶段	状态机定义	值	释义
冷启动阶段	STATE_INVALID	0	无效状态
	STATE_WAIT_FOR_VHAL	1	CarPower启动阶段首个状态，等待连接hal。
STR第一阶段(SHUTDOWN_PREPARE 阶段)	STATE_PRE_SHUTDOWN_PREPARE	11	Android收到STR指令后，开始执行 SHUTDOWN_PREPARE 阶段的第一个子状态。 应用层在该状态下需要做资源释放，以免休眠失败。
	STATE_SHUTDOWN_PREPARE	7	Android收到STR指令后，开始执行 SHUTDOWN_PREPARE 阶段的第二个子状态。 应用层在该状态下需要做资源释放，以免休眠失败。
STR第二阶段(WAIT_FOR_FINISH 阶段)	STATE_SUSPEND_ENTER	2	Android收到STR指令后的 WAIT_FOR_FINISH 阶段。 根据STR指令参数，准备执行 suspend to ram 休眠流程。

			该状态通知后，Android开始准备执行第三阶段 kernel suspend to ram。
	STATE_SHUTDOWN_ENTER	5	Android收到STR指令后的WAIT_FOR_FINISH 阶段。 根据STR指令参数，准备执行shutdown关机流程。 该状态通知后，Android开始准备执行第三阶段关机操作。
	STATE_HIBERNATION_ENTER	9	Android收到STR指令后的WAIT_FOR_FINISH 阶段。 根据STR指令参数，准备执行hibernation 休眠流程。 该状态通知后，Android开始准备执行第三阶段 kernel suspend to disk。
STR第三阶段(SUSPEND 阶段)	STATE_POST_SUSPEND_ENTER	12	Android收到STR指令后的SUSPEND阶段。 根据STR指令参数，开始执行 str 休眠 kernel suspend to ram，在执行之前先通知该状态。
	STATE_POST_SHUTDOWN_ENTER	13	Android收到STR指令后的SUSPEND阶段。 根据STR指令参数，开始执行关机操作，在此执行关机之前先通知该状态。
	STATE_POST_HIBERNATION_ENTER	14	Android收到STR指令后的SUSPEND阶段。 根据STR指令参数，开始执行hibernation 休眠 kernel suspend to disk，在执行之前先通知该状态。
唤醒阶段	STATE_SUSPEND_EXIT	3	Android从 suspend to ram 休眠模式下退出。
	STATE_SHUTDOWN_CANCELLED	8	Android收到取消STR指令后，执行cancel流程，并通知应用层。

	STATE_HIBERNATION_EXIT	10	Android从hibernation休眠模式下退出。
Android完全工作阶段	STATE_ON	6	Android处于工作的active活跃状态，此状态机下屏幕通常处于点亮状态。
	STATE_STANDBY	15	Android处于待机状态，该状态机下屏幕一般处于灭屏状态(具体表现根据项目需求决定)。
	STATE_SCREEN_SAVER	16	Android处于屏保状态。
	STATE_TEMPORARY_WORKING	17	Android处于临时工作状态。
	STATE_REMOTE_BOOT	18	Android处于远程启动状态。
	STATE_OVER_VOLTAGE	19	Android处于高电压异常状态。
	STATE_LOW_VOLTAGE	20	Android处于低电压异常状态。
	STATE_ABANDONED	21	Android处于Abandoned状态(岚图项目独有，其表现和STANDBY基本一致)。
	STATE_BOOT_ANIMATION	22	Android处于开机动画状态。
	STATE_OTA	23	Android处于OTA升级状态。
	STATE_FACTORY_MODE	24	奇瑞项目特有:工厂模式
	STATE_FACTORY_PAUSED_MODE	25	奇瑞项目特有:工厂暂停模式
	STATE_TRANSPORT_MODE	26	奇瑞项目特有:运输模式
	STATE_TRANSPORT_PAUSED_MODE	27	奇瑞项目特有:运输暂停模式
	STATE_CRASH_MODE	28	奇瑞项目特有:CRASH模式
	STATE_DYNO_MODE	29	奇瑞项目特有:DYNO模式
	STATE_HIGH_TEMPERATURE	30	预留:高温异常模式
	STATE_CHARGE_1MIN	31	一分钟充电状态
	STATE_BACKUP_BAT	32	备电状态

注:在8155平台Android 11上Google CarPowerManager中的电源状态机是定义在CarPowerManager#CarPowerStateListener中的，Android 13上Google将其调整到了CarPowerManager#下，为了应用层app快速方便对接从Android 11 升级到 Andriod 13，我们在

Android 13 CarPowerManager#CarPowerStateListener下补充回来了电源状态机的定义(这里不具体解释状态机的定义，value和释义没区别）。
但是建议应用层最好是使用CarPowerManager#下的定义，因为这是Google的标准做法。

2.2 SettingsProvider中的电源状态机定义

为了方便一些三方app少集成平台sdk，如果app不希望集成android.car.jar，也想要对接电源管理。我们镁佳平台提供了一套基于Android原生ContentObserver的监听方式来获取电源状态机。区别于Android 11，在Android 13、14上，Settings侧的电源状态机其value值保持和CarPowerManager中一致，具体定义如下：

阶段	Settings.System中定义	值	释义
冷启动阶段	STATUS_INVALID	0	无效状态
	STATUS_WAIT_FOR_VHAL	1	CarPower启动阶段首个状态，等待连接hal。
STR第一阶段(SHUTDOWN_PREPARE 阶段)	STATUS_PRE_SHUTDOWN_PREPARE	11	Android收到STR指令后，开始执行 SHUTDOWN_PREPARE 阶段的第一个子状态。 应用层在该状态下需要做资源释放，以免休眠失败。
	STATUS_SHUTDOWN_PREPARE	7	Android收到STR指令后，开始执行 SHUTDOWN_PREPARE 阶段的第二个子状态。 应用层在该状态下需要做资源释放，以免休眠失败。
STR第二阶段(WAIT_FOR_FINISH 阶段)	STATUS_SUSPEND_ENTER	2	Android收到STR指令后的WAIT_FOR_FINISH 阶段。 根据STR指令参数，准备执行suspend to ram 休眠流程。 该状态通知后，Android开始准备执行第三阶段 kernel suspend to ram。
	STATUS_SHUTDOWN_ENTER	5	Android收到STR指令后的WAIT_FOR_FINISH 阶段。 根据STR指令参数，准备执行shutdown关机流程。 该状态通知后，Android开始准备

			执行第三阶段关机操作。
	STATUS_HIBERNATION_ENTER	9	Android收到STR指令后的WAIT_FOR_FINISH 阶段。 根据STR指令参数，准备执行hibernation 休眠流程。 该状态通知后，Android开始准备执行第三阶段 kernel suspend to disk。
STR第三阶段(SUSPEND 阶段)	STATUS_POST_SUSPEND_ENTER	12	Android收到STR指令后的SUSPEND阶段。 根据STR指令参数，开始执行 str 休眠 kernel suspend to ram，在执行之前先通知该状态。
	STATUS_POST_SHUTDOWN_ENTER	13	Android收到STR指令后的SUSPEND阶段。 根据STR指令参数，开始执行关机操作，在此执行关机之前先通知该状态。
	STATUS_POST_HIBERNATION_ENTER	14	Android收到STR指令后的SUSPEND阶段。 根据STR指令参数，开始执行hibernation 休眠 kernel suspend to disk，在执行之前先通知该状态。
唤醒阶段	STATUS_SUSPEND_EXIT	3	Android从 suspend to ram 休眠模式下退出。
	STATUS_SHUTDOWN_CANCELLED	8	Android收到取消STR指令后，执行cancel流程，并通知应用层。
	STATUS_HIBERNATION_EXIT	10	Android从hibernation休眠模式下退出。
Android工作阶段	STATUS_ON	6	Android处于工作的active活跃状态，此状态机下屏幕通常处于点亮状态。
	STATUS_STANDBY	15	Android处于待机状态，该状态机下屏幕一般处于灭屏状态(具体表现根据项目需求决定)。

	STATUS_SCREEN_SAVER	16	Android处于屏保状态。
	STATUS_TEMPORARY_WORKING	17	Android处于临时工作状态。
	STATUS_REMOTE_BOOT	18	Android处于远程启动状态。
	STATUS_OVER_VOLTAGE	19	Android处于高电压异常状态。
	STATUS_LOW_VOLTAGE	20	Android处于低电压异常状态。
	STATUS_ABANDONED	21	Android处于Abandoned状态(岚图项目独有，其表现和STANDBY基本一致)。
	STATUS_BOOT_ANIMATION	22	Android处于开机动画状态。
	STATUS_OTA	23	Android处于OTA升级状态。
	STATUS_FACTORY	24	奇瑞项目特有:工厂模式
	STATUS_FACTORY_PAUSED	25	奇瑞项目特有:工厂暂停模式
	STATUS_TRANSPORT	26	奇瑞项目特有:运输模式
	STATUS_TRANSPORT_PAUSED	27	奇瑞项目特有:运输暂停模式
	STATUS_CRASH	28	奇瑞项目特有:CRASH模式
	STATUS_DYNO	29	奇瑞项目特有:DYNO模式
	STATUS_HIGH_TEMPERATURE	30	预留:高温异常模式
	STATUS_CHARGE_1MIN	31	一分钟充电状态
	STATUS_BACKUP_BAT	32	备电状态

3、对上层应用的接口

3.1 Google原生AndroidCar接口

使用Google原生Car相关接口需要集成8295平台Android 13发布的android.car.jar，该jar由项目负责对外提供。

3.1.1 连接AndroidCar服务

3.1.1.1 Car.java

Public methods	
static Car	createCar(Context context, Handler handler, long waitTimeoutMs, Car.CarServiceLifecycleListener statusChangeListener)
	使用 CarServiceLifecycleListener 方式连接创建AndroidCar服务。 建议使用该接口连接AndroidCar服务，该方式会在AndroidCar ready或者died时回调通知，不会导致因AndroidCar服务崩溃引发关联的App进程被杀。

Public methods	
Object	getCarManager(String serviceName)
	获取AndroidCar中提供的Manager

3.1.1.2 Car.CarServiceLifecycleListener

Nested classes	
interface	Car.CarServiceLifecycleListener
	回调通知AndroidCar服务的生命周期
Public methods	
abstract void	onLifecycleChanged(@NonNull Car car, boolean ready)
	AndroidCar服务状态变化

3.1.2 CarPowerManager

电源状态机的定义在 第二章节电源状态机 中已做介绍，这里不再赘述。

3.1.2.1 CarPowerManager.java

Public methods	
int	getPowerState()
	获取当前Android侧电源状态机(定义在CarPowerManager侧的int值) 需要权限：android.car.permission.CAR_POWER

Public methods	
void	setListener(@NonNull @CallbackExecutor Executor executor, @NonNull CarPowerStateListener listener)
	使用 Executor、CarPowerStateListener 注册电源状态机监听。 Executor 指定 CarPowerStateListener 回调所运行的线程。 需要权限：android.car.permission.CAR_POWER

Public methods	
void	setListener(@NonNull CarPowerStateListener listener)
	使用 CarPowerStateListener 注册电源状态机监听。 不指定 Executor，则 CarPowerStateListener 回调默认执行在主线程。 需要权限：android.car.permission.CAR_POWER (该**API**为镁佳平台在**Android 13**上补充，方便**app**从**Android 11**迁移升级到**Android 13**时尽量少调整代码。但是建议**app**注册状态机监听时指定**Executor**)

Public methods	
void	setListenerWithCompletion(@NonNull @CallbackExecutor Executor executor, @NonNull CarPowerStateListenerWithCompletion listener)
	使用 Executor、CarPowerStateListenerWithCompletion 注册电源状态机监听。 Executor 指定 CarPowerStateListenerWithCompletion 回调所运行的线程。 需要权限：android.car.permission.CONTROL_SHUTDOWN_PROCESS

Public methods	
void	setListenerWithCompletion(@NonNull CarPowerStateListenerWithCompletion listener)
	使用 CarPowerStateListenerWithCompletion 注册电源状态机监听。 不指定 Executor，则 CarPowerStateListenerWithCompletion 回调默认执行在主线程。 需要权限：android.car.permission.CONTROL_SHUTDOWN_PROCESS (该**API**为镁佳平台在**Android 13**上补充，方便**app**从**Android 11**迁移升级到**Android 13**时尽量少调整代码。但是建议**app**注册状态机监听时指定**Executor**)

Public methods	
void	clearListener()
	解注册电源状态机监听

Public methods	
void	addPowerPolicyListener(@NonNull @CallbackExecutor Executor executor, @NonNull CarPowerPolicyFilter filter, @NonNull CarPowerPolicyListener listener)
	注册CarPowerPolicy监听Listener。 通过该API，应用可以监听配置在/vendor/etc/automotive/power_policy.xml中各个电源状态机下不同组件的policy策略。 当电源状态机切换，且在该状态机下配置有相关组件的policy策略，CarPowerManagementService则会向监听端CarPowerPolicyListener通知当前的CarPowerPolicy。 需要权限：android.car.permission.READ_CAR_POWER_POLICY

Public methods	
void	removePowerPolicyListener(@NonNull CarPowerPolicyListener listener)
	移除注册的CarPowerPolicy监听Listener。

	需要权限：android.car.permission.READ_CAR_POWER_POLICY

Public methods	
CarPowerPolicy	getCurrentPowerPolicy()
	获取当前的 PowerPolicy。 如果当前电源状态机没有在power_policy.xml中配置相关policy，则获取到的是上次执行的PowerPolicy。 需要权限：android.car.permission.READ_CAR_POWER_POLICY

依赖权限：

Permission	ProtectionLevel
android.car.permission.CAR_POWER	signature | privileged
android.car.permission.CONTROL_SHUTDOWN_PROCESS	signature | privileged
android.car.permission.READ_CAR_POWER_POLICY	normal

3.1.2.2 CarPowerManager.CarPowerStateListener

Nested classes	
interface	CarPowerManager.CarPowerStateListener
	电源状态机变化回调
Public methods	
abstract void	onStateChanged(@CarPowerState int state)
	电源状态机变化

3.1.2.3 CarPowerManager.CarPowerStateListenerWithCompletion

Nested classes	
interface	CarPowerManager.CarPowerStateListenerWithCompletion
	电源状态机变化回调

Public methods	
abstract void	onStateChanged(@CarPowerState int state, @Nullable CompletablePowerStateChangeFuture future)
	电源状态机变化。 注: 使用该 Listener 监听电源状态机时，需要在回调参数 future 不为 null 时调用 **future.complete()** 通知CarPower解除阻塞，否则会导致CarPower被阻塞无法执行后续流程。

3.1.2.4 CarPowerManager.CarPowerPolicyListener

Nested classes	
interface	CarPowerManager.CarPowerPolicyListener
	电源状态机下配置的CarPowerPolicy
Public methods	
abstract void	onPolicyChanged(@NonNull CarPowerPolicy)
	电源状态机下配置的CarPowerPolicy变化回调。 当电源状态机切换，且在该状态机下配置有相关组件的policy策略，CarPowerManagementService则会通过该回调接口向监听端CarPowerPolicyListener通知当前的CarPowerPolicy。

3.1.3 CarPowerPolicy

Public methods	
int[]	getDisabledComponents()
	获取当前CarPowerPolicy中配置为off，做disable处理的component集合
int[]	getEnabledComponents()
	获取当前CarPowerPolicy中配置为off，做disable处理的component集合
String	getPolicyId()
	获取当前CarPowerPolicy的Policy Id

boolean	isComponentEnabled(int component)
	检查指定component是否配置policy为on。 **(注：该Android原生API实现不合理。 Google在设计 CarPowerPolicy 时对于component的处理有三种配置:on、off、untouched。 而该API的实现逻辑为：只要不是policy on 就返回false。 因此不建议使用该原生API来作为enable、disable component的依据。 可以使用镁佳平台增加的API getPolicyStateForComponent 来获取component的真实policy配置)**
int[]	getUntouchedComponents()
	获取当前CarPowerPolicy中配置为untouched，保持当前状态无需处理的component 集合。 (该API为镁佳平台增加)
int	getPolicyStateForComponent(int component)
	获取当前CarPowerPolicy中component的policy状态。 (该API为镁佳平台增加)

Constants	
int	POLICY_ON
	表示power_policy.xml中配置的该组件策略为on，即做enable处理
int	POLICY_OFF
	表示power_policy.xml中配置的该组件策略为off，即做disable处理
int	POLICY_UNTOUCHED
	表示power_policy.xml中配置的该组件策略为untouched，即保持当前状态，不做处理。

3.1.4 CarPowerPolicyFilter

Nested classes	
class	CarPowerPolicyFilter.Builder

	Builder for CarPowerPolicyFilter

Public methods	
int[]	getComponents()
	获取Filter中设置监听的组件components 集合

3.1.4.1 CarPowerPolicyFilter.Builder

Public methods	
CarPowerPolicyFilter	build()
	Build CarPowerPolicyFilter 实例
CarPowerPolicyFilter.Builder	setComponents(@NonNull int… components)
	设置需要监听policy变化的组件 component 集。 支持的组件Component 见 PowerComponent。

3.1.5 PowerComponent

Constants	
int	AUDIO
int	MEDIA
int	DISPLAY
int	BLUETOOTH
int	WIFI
int	CELLULAR
int	ETHERNET
int	PROJECTION
int	NFC

int	INPUT
int	VOICE_INTERACTION
int	VISUAL_INTERACTION
int	TRUSTED_DEVICE_DETECTION
int	LOCATION
int	MICROPHONE
int	CPU
以下为镁佳平台增加	
int	PASSENGER_DISPLAY
int	VEHICLE_DISPLAY
int	CEILING_DISPLAY
int	LEFT_REAR_DISPLAY
int	RIGHT_REAR_DISPLAY
int	MID_REAR_DISPLAY
int	PASSENGER_INPUT
int	VEHICLE_INPUT
int	CEILING_INPUT
int	LEFT_REAR_INPUT
int	RIGHT_REAR_INPUT
int	MID_REAR_INPUT
int	WIFI_AP
int	USB
int	CAMERA
int	PROCESS

3.2 镁佳ContentObserver方式监听电源状态机变化

镁佳平台增加的使用 ContentObserver 监听电源状态机的方式依赖于Android原生的ContentProvider机制，不需要额外集成sdk。
但由于Android版本变化，状态机的定义有可能会有区别，因此对于电源状态机的定义和Content Uri还是建议使用**mega.nexus.framework.static.source.jar**中的定义，static jar中区分了Android 11 和 Android 13中的差异。

在Android 11 平台，电源状态机的定义使用MegaSettings.java中的定义。
在Android 13 平台，电源状态机的定义使用PowerSettings13.java中的定义(第二章节 SettingsProvider中的电源状态机定义 已做介绍，不再赘述)。

mega.nexus.framework.static.source.jar 由项目提供。

Demo参考: MegaCarPowerDemo13.java

3.3 Android侧强行切换整机电源状态机

在项目需求的实施过程中，部分状态机的切换条件判定是交给业务层来处理的。因此有一些场景，当满足状态机切换的条件时，状态机的切换不是QNX侧power主动发起，而是由Android侧的业务模块通知QNX侧power来强行整机切换电源状态机。

Android侧发起强行切换电源管理状态机的方式有两种：
1、通过CarPowerManager提供的接口来通知QNX侧power强行切换状态机。接口如下：

Public methods	
void	requestToSwitchCarPowerState(@CarPowerState int targetState, @NonNull String reason)
	targetState 为目标切换的电源状态机。 reason 为切换状态机的原因(便于问题排查)。 需要权限：android.car.permission.CAR_POWER

2、通过调用MegaCar相关接口直接发topic信号的方式，来通知QNX侧power强行切换状态机。
topic信号如下：

Class	mega.car.config.Power

Constants

int	@PropertyDefine(publish = "Power/AndroidStatusReport", type = ReportEntry.class, changeMode = CarPropertyConfig.VEHICLE_PROPERTY_CHANGE_MODE_CONTINUOUS) ANDROID_STATUS_REPORT
	Power/AndroidStatusReport 为信号topic字符 ReportEntry 为信号value数据 需要权限：android.car.permission.CAR_POWER

Class	mega.car.config.Power.ReportEntry

Public constructors
ReportEntry(String status, int syncDirection, String reason)
status: 目标切换的电源状态机。传参时使用 mega.car.config.Power.PowerStatusEnum 中定义的值。 syncDirection: 同步类型。强行切换状态机时，传入 0。 reason: 强行切换电源状态机的原因(以便排查问题)。

4、对Android 应用层app的要求

在车载电源管理中，对app的要求主要是在str阶段进行资源释放，避免因资源未释放导致休眠失败。
电源管理中str阶段涉及到的状态机均在 第二章节电源状态机 中标红。
对于应用来讲，一般要求在 STATE_SHUTDOWN_PREPARE 或 STATE_PRE_SHUTDOWN_PREPARE 这两个阶段就要做资源释放，不要晚于该阶段。
其次就是 CarPowerManager.CarPowerStateListenerWithCompletion 的使用注意，也已标红。